E4 Educator v2.0

Tier 3 · Project 1 of 5

P01

Environmental Monitor

Walark Electronics · Fundrum Engineering · May 2026

Welcome to Tier 3 — Integrated Projects

Tier 3 projects combine everything from Tier 1 and Tier 2 into complete, deployable, real-world firmware. There are no new concepts to learn — only new ways to combine what you already know. Each project is specification-driven, version-controlled, and built to Fundrum production standards.

Project overview

The Environmental Monitor is a fully self-contained sensor station that reads temperature, humidity, and light level, displays live readings on the TFT dashboard, logs data to SD card, publishes to MQTT, serves a web dashboard via LittleFS, stores calibrated thresholds in NVS, and supports OTA firmware updates. It operates continuously without a USB cable.

Skills combined from:

T01 millis() timing · T02 GPIO/LEDC · T03 Button.h · T04 I2C/AHT20 · T06 DS18B20/LDR · T07 TonePlayer.h · T08 TFT sprite · T09 TouchZone.h · T10 SD logging · T11 NVS/Settings.h · T12 WiFi/MQTT · T13 LittleFS web · T14 OTA

1 Project specification

Define the complete specification before writing a single line of code. This is the Fundrum discipline — the spec is the contract between what you plan to build and what you actually build.

1.1 Feature specification

ID	Feature	Detail
F01	AHT20 sensor	Temperature (±0.3°C) and humidity (±2%) read every 5 seconds
F02	DS18B20 probe	External probe temperature, non-blocking 12-bit conversion, displayed alongside AHT20
F03	LDR light sensor	8-sample averaged ADC1 read, 0-100% scale, bar graph on display
F04	TFT dashboard	Three-zone portrait layout. Sprite pipeline. Colour-coded values. Two pages: live readings and statistics.
F05	Touch controls	Footer touch zones: REFRESH, PAGE, SETTINGS. Physical NEXT and ENT buttons also functional.
F06	SD data logger	CSV log: millis, tempAHT, tempDS18, humidity, lightPct. Rotation at 512KB. On/off toggle via ENT.
F07	NVS settings	tempAlert, humidAlert, brightness, logInterval stored via Settings.h. Factory reset on dual-button boot.
F08	WiFi + MQTT	Non-blocking state machine. Publishes all sensor values retained. LWT e4/status online/offline.
F09	LittleFS web server	Serves index.html dashboard. /api/sensors JSON endpoint. /api/settings GET/POST.
F10	OTA firmware update	ElegantOTA at /update. TFT progress bar. Firmware version displayed in header and published to MQTT.
F11	Alert system	TonePlayer.h alert when temp or humidity exceeds NVS threshold. ENT acknowledges. BLUE LED active.
F12	Status LEDs	GREEN = running OK. RED = sensor error. BLUE = alert active or fast mode.

1.2 Hardware requirements

Component	Detail
E4 Educator v2.0	ESP32 DevKit V1 socketed, TFT module fitted, OLED optional
microSD card	FAT32 formatted, 2-32GB, Class 10 recommended
Power supply	USB-C 5V minimum 500mA. For standalone: USB power bank or 5V/1A wall adapter.
WiFi network	2.4GHz WPA2. MarkPi3 Mosquitto broker at 192.168.1.79 for MQTT.

1.3 Version information

Version	Date	Author	Changes
1.0.0	May 2026	Mark Weir	Initial release

2 Architecture

2.1 Project file structure

```
E4_P01_EnvMonitor/
├── platformio.ini
├── src/
│   ├── main.cpp          ← application entry, loop(), state
│   ├── Button.h          ← copy from T03
│   ├── TonePlayer.h      ← copy from T07
│   ├── TouchZone.h       ← copy from T09
│   ├── Settings.h        ← copy from T11
│   ├── Sensors.h         ← NEW: all sensor read/state functions
│   ├── Display.h         ← NEW: all TFT drawing functions
│   ├── Network.h         ← NEW: WiFi + MQTT + web server
│   └── Logger.h          ← NEW: SD card CSV logging
└── data/
    ├── index.html        ← web dashboard
    └── config.json       ← default configuration
```

Key point

Tier 3 projects split across multiple header files rather than a single large main.cpp. Each header handles one domain: sensors, display, network, logging. main.cpp becomes a clean coordinator that calls into these modules. This is the pattern used in commercial embedded firmware.

2.2 Subsystem interactions

Subsystem	Depends on / Interacts with
Sensors.h	Wire (I2C), OneWire, analogRead, millis() state machine for DS18B20
Display.h	TFT_eSPI, TFT_eSprite, Button.h, TouchZone.h, sensor values from Sensors.h
Network.h	WiFi, PubSubClient, ESPAsyncWebServer, LittleFS, ElegantOTA, sensor values
Logger.h	SD, sensor values, millis() for timestamps
Settings.h	Preferences (NVS). Read by all subsystems. Written by Display.h settings page.
main.cpp	Calls all subsystems. Owns global state. Runs timed coordinator loop.

2.3 loop() structure

The main loop is a flat coordinator. No blocking operations. No nested logic. Every subsystem call is a named function with a clear purpose:

```
void loop() {
  unsigned long now = millis();

  // ── Non-blocking subsystem heartbeats ──────────────────
  wifiUpdate(now);          // WiFi state machine
  mqttUpdate(now);          // MQTT reconnect + client.loop()
```

```
  ElegantOTA.loop();        // OTA receive
  sensorsUpdate(now);       // DS18B20 state machine tick

  // ── Button and touch events ──────────────────────────
  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();
  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);

  handleButtons(nextEv, entEv);
  handleTouch(tx, ty, touched);
  tonePlayer.update();      // Non-blocking audio

  // ── Timed sensor read cycle ──────────────────────────
  if (now - lastSensorRead >= settings.s.logInterval) {
    lastSensorRead = now;
    readAllSensors();       // AHT20 + LDR (DS18B20 async)
    checkAlerts();
    updateDisplay();
    logReading();
    publishSensors();
  }

  // ── Footer clock update ──────────────────────────────
  if (now - lastFooter >= 1000) {
    lastFooter = now;
    updateFooter(now);
  }

  // ── OTA validity mark ────────────────────────────────
  if (!otaValid && now > 10000) {
    esp_ota_mark_app_valid_cancel_rollback();
    otaValid = true;
  }
}
```

3 TFT dashboard design

3.1 Zone layout — portrait 240×320

Zone	Y range	Height	Content
Header	Y 0–34	35px	Project title, FW version, WiFi/MQTT status dots
Page area	Y 35–199	165px	Sensor values (page 1) or statistics (page 2)
Footer	Y 200–239	40px	Touch buttons: REFRESH · PAGE · SETTINGS. Uptime clock.

3.2 Page 1 — live readings layout

Sub-zone	Y range	Content and colour logic
AHT20 temp	Y 43–74	Label + large value. GREEN <24°C, YELLOW 24–28°C, RED >28°C
Divider	Y 95	drawFastHLine() COL_LABEL
AHT20 humidity	Y 78–114	Label + large value. GREEN <60%, YELLOW 60–70%, RED >70%
Divider	Y 145	drawFastHLine() COL_LABEL
DS18B20 probe	Y 118–154	Label + value. Same colour logic as AHT20 temp. —.— if not yet read.
Divider	Y 195	drawFastHLine() COL_LABEL
Light bar	Y 158–196	Label + percentage + horizontal bar. ACCENT colour fill.

3.3 Page 2 — statistics layout

Page 2 shows accumulated min/max values, reading count, uptime, log file status, and alert threshold settings. This gives the operator a historical view without leaving the display.

Row	Y range	Content
Temp stats	Y 55–85	Min: xx.x Max: xx.x °C (AHT20)
Humid stats	Y 88–118	Min: xx.x Max: xx.x %
Reading count	Y 125–145	Readings: NNNN Up: NNNs
Log status	Y 150–170	SD: OK/FAIL Rows: NNNN Size: NNKb
Thresholds	Y 178–220	Alert T>xx.x°C H>xx.x% Interval: Ns
Network	Y 225–270	WiFi: OK/... MQTT: OK/... IP: xxx.xxx.xxx.xxx

4 Display.h — TFT drawing module

```
// Display.h — E4 P01 Environmental Monitor
#pragma once
#include <Arduino.h>
#include <TFT_eSPI.h>
#include "TouchZone.h"
#include "Settings.h"

// ── Colour palette ──────────────────────────────────────────
#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_FOOTER  0x18C3
#define COL_TEXT    0xFFFF
#define COL_LABEL   0xC618
#define COL_GRID    0x1082
#define COL_OK      0x07E0
#define COL_WARN    0xFFE0
#define COL_ERR     0xF800
#define COL_ACCENT  0xFD20

extern TFT_eSPI    tft;
extern TFT_eSprite spr;
extern SettingsManager settings;

// ── Footer touch zones ──────────────────────────────────────
TouchZone tzRefresh ( 4,  202, 72, 36, "REFRESH");
TouchZone tzPage    (84,  202, 72, 36, "PAGE");
TouchZone tzSettings(164, 202, 72, 36, "SET");

// ── Zone constants ──────────────────────────────────────────
#define ZONE_HDR_Y     0
#define ZONE_HDR_H    35
#define ZONE_CONTENT_Y 40
#define ZONE_CONTENT_H 165
#define ZONE_FTR_Y   200
#define ZONE_FTR_H    40

enum DisplayPage { PAGE_LIVE, PAGE_STATS, PAGE_SETTINGS, PAGE_COUNT };
DisplayPage currentPage = PAGE_LIVE;

uint16_t tempColour(float t, float alertHi) {
  if (t > alertHi)         return COL_ERR;
  if (t > alertHi - 4.0f) return COL_WARN;
  return COL_OK;
}

uint16_t humidColour(float h, float alertHi) {
  if (h > alertHi)          return COL_ERR;
  if (h > alertHi - 10.0f) return COL_WARN;
  return COL_OK;
}
```

```
void drawChrome() {
  tft.fillScreen(COL_BG);
  // Header
  tft.fillRect(0, ZONE_HDR_Y, SCR_W, ZONE_HDR_H, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("Env Monitor");
  // Static dividers
  tft.drawFastHLine(0,  62, SCR_W, COL_LABEL);
  tft.drawFastHLine(0,  96, SCR_W, COL_LABEL);
  tft.drawFastHLine(0, 130, SCR_W, COL_LABEL);
  // Static labels
  tft.setTextSize(1);
  tft.setTextColor(COL_LABEL, COL_BG);
  tft.setCursor(8, 40);  tft.print("AIR TEMP");
  tft.setCursor(8, 68);  tft.print("HUMIDITY");
  tft.setCursor(8, 100); tft.print("PROBE TEMP");
  tft.setCursor(8, 136); tft.print("LIGHT");
}

void updateLivePage(float ahtTemp, float ahtHumid,
                    float ds18Temp, int lightPct,
                    float tAlert, float hAlert) {
  // ── Air temperature zone ──────────────────────────
  spr.fillSprite(COL_BG);
  spr.setTextColor(tempColour(ahtTemp, tAlert), COL_BG);
  spr.setTextSize(4);
  spr.setCursor(8, 6);
  spr.printf("%.1f C", ahtTemp);
  spr.pushSprite(0, 43);

  // ── Humidity zone ─────────────────────────────────
  spr.fillSprite(COL_BG);
  spr.setTextColor(humidColour(ahtHumid, hAlert), COL_BG);
  spr.setTextSize(4);
  spr.setCursor(8, 6);
  spr.printf("%.1f %%", ahtHumid);
  spr.pushSprite(0, 75);

  // ── DS18B20 probe zone ────────────────────────────
  spr.fillSprite(COL_BG);
  spr.setTextColor(tempColour(ds18Temp, tAlert), COL_BG);
  spr.setTextSize(4);
  spr.setCursor(8, 6);
  if (ds18Temp > -90.0f)
    spr.printf("%.1f C", ds18Temp);
  else
    spr.print("--.- C");
  spr.pushSprite(0, 106);

  // ── Light bar zone ────────────────────────────────
  spr.fillSprite(COL_BG);
  spr.setTextColor(COL_TEXT, COL_BG);
```

```
  spr.setTextSize(2);
  spr.setCursor(8, 4);
  spr.printf("%d %%", lightPct);
  spr.drawRect(0, 30, 230, 16, COL_LABEL);
  int fw = map(lightPct, 0, 100, 0, 228);
  if (fw > 0) spr.fillRect(1, 31, fw, 14, COL_ACCENT);
  spr.pushSprite(8, 138);
}

void updateFooter(unsigned long now,
                  bool wifiOk, bool mqttOk,
                  bool logOn) {
  spr.fillSprite(COL_FOOTER);

  // Status dots
  spr.fillCircle(8,  8, 4, wifiOk ? COL_OK : COL_ERR);
  spr.fillCircle(18, 8, 4, mqttOk ? COL_OK : COL_ERR);
  spr.fillCircle(28, 8, 4, logOn  ? COL_OK : COL_WARN);

  // Uptime
  spr.setTextColor(COL_LABEL, COL_FOOTER);
  spr.setTextSize(1);
  spr.setCursor(38, 4);
  spr.printf("Up:%lus", now / 1000);

  // Touch buttons
  tzRefresh.draw(spr, COL_HEADER, COL_TEXT, COL_TEXT, 0, 280);
  tzPage.draw   (spr, COL_HEADER, COL_TEXT, COL_TEXT, 0, 280);
  tzSettings.draw(spr,COL_HEADER, COL_TEXT, COL_TEXT, 0, 280);

  spr.pushSprite(0, ZONE_FTR_Y);
}

void handleTouch(uint16_t tx, uint16_t ty, bool touched,
                 bool& forceRead, unsigned long now) {
  if (tzRefresh.isTapped(tx, ty, touched)) forceRead = true;
  if (tzPage.isTapped(tx, ty, touched)) {
    currentPage = (DisplayPage)((currentPage + 1) % PAGE_COUNT);
  }
  if (tzSettings.isTapped(tx, ty, touched))
    currentPage = PAGE_SETTINGS;
}
```

5 Sensors.h — sensor reading module

```
// Sensors.h — E4 P01 Environmental Monitor
#pragma once
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include <OneWire.h>
#include <DallasTemperature.h>

Adafruit_AHTX0  aht;
OneWire          ow(ONE_WIRE);
DallasTemperature ds(&ow);

// Published sensor values (read by Display, Network, Logger)
float ahtTempC   = 0.0f;
float ahtHumid   = 0.0f;
float ds18TempC  = -99.0f;  // -99 = not yet read
int   lightPct   = 0;
float minTemp = 999.0f, maxTemp = -999.0f;
float minHumid= 999.0f, maxHumid= -999.0f;
long  readCount  = 0;
bool  sensorsOk  = false;

// DS18B20 non-blocking state
enum DS18St { DS18_IDLE, DS18_CONVERTING };
DS18St ds18State    = DS18_IDLE;
unsigned long ds18Start = 0;

bool initSensors() {
  Wire.begin(I2C_SDA, I2C_SCL);
  if (!aht.begin(&Wire)) {
    Serial.println("AHT20 not found"); return false;
  }
  ds.begin();
  ds.setResolution(12);
  ds.setWaitForConversion(false);
  Serial.printf("Sensors: AHT20 OK, DS18B20: %d found\n",
                ds.getDeviceCount());
  return true;
}

int readLDR() {
  long t = 0;
  for (int i = 0; i < 8; i++) { t += analogRead(LDR); delay(2); }
  return constrain(map(t/8, 200, 3800, 0, 100), 0, 100);
}

// Call every loop() pass for non-blocking DS18B20
void sensorsUpdate(unsigned long now) {
  switch (ds18State) {
    case DS18_IDLE:
      ds.requestTemperatures();
      ds18Start = now;
```

```
      ds18State = DS18_CONVERTING;
      break;
    case DS18_CONVERTING:
      if (now - ds18Start >= 750) {
        float t = ds.getTempCByIndex(0);
        if (t != DEVICE_DISCONNECTED_C) ds18TempC = t;
        ds18State = DS18_IDLE;
      }
      break;
  }
}

void readAllSensors() {
  sensors_event_t h, t;
  aht.getEvent(&h, &t);
  ahtTempC  = t.temperature;
  ahtHumid  = h.relative_humidity;
  lightPct  = readLDR();
  readCount++;
  // Update statistics
  if (ahtTempC < minTemp)  minTemp  = ahtTempC;
  if (ahtTempC > maxTemp)  maxTemp  = ahtTempC;
  if (ahtHumid < minHumid) minHumid = ahtHumid;
  if (ahtHumid > maxHumid) maxHumid = ahtHumid;
  Serial.printf("T:%.1f H:%.1f L:%d P:%.1f reads:%ld\n",
                ahtTempC, ahtHumid, lightPct, ds18TempC, readCount);
}
```

6 Network.h — WiFi, MQTT, and web server

```
// Network.h — E4 P01 Environmental Monitor
#pragma once
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClient.h>
#include <PubSubClient.h>
#include <LittleFS.h>
#include <ESPAsyncWebServer.h>
#include <ElegantOTA.h>
#include "Settings.h"

// Declared in Sensors.h — extern to avoid redeclaration
extern float ahtTempC, ahtHumid, ds18TempC;
extern int   lightPct;
extern long  readCount;

WiFiClient     wifiClient;
PubSubClient   mqtt(wifiClient);
AsyncWebServer server(80);

// ── Network credentials from NVS ──────────────────────────
String wifiSSID, wifiPass, mqttBroker;

// ── WiFi state machine ────────────────────────────────────
enum WiFiSt { WIFI_IDLE, WIFI_CONNECTING, WIFI_CONNECTED, WIFI_FAILED };
WiFiSt wifiSt = WIFI_IDLE;
unsigned long wifiT = 0;

bool wifiReady() { return wifiSt == WIFI_CONNECTED; }

void wifiConnect() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(wifiSSID.c_str(), wifiPass.c_str());
  wifiT = millis(); wifiSt = WIFI_CONNECTING;
  Serial.printf("Connecting to %s...\n", wifiSSID.c_str());
}

void wifiUpdate(unsigned long now) {
  switch (wifiSt) {
    case WIFI_CONNECTING:
      if (WiFi.status() == WL_CONNECTED) {
        wifiSt = WIFI_CONNECTED;
        Serial.printf("WiFi OK. IP: %s\n",
                      WiFi.localIP().toString().c_str());
      } else if (now - wifiT > 15000) {
        wifiSt = WIFI_FAILED; wifiT = now;
      }
      break;
    case WIFI_CONNECTED:
      if (WiFi.status() != WL_CONNECTED) {
        wifiSt = WIFI_CONNECTING; wifiT = now; WiFi.reconnect();
      }
```

```
      break;
    case WIFI_FAILED:
      if (now - wifiT > 30000) wifiConnect();
      break;
    default: break;
  }
}

// ── MQTT ──────────────────────────────────────────────────
void mqttCallback(char* topic, byte* payload, unsigned int len) {
  String msg; for (unsigned int i=0; i<len; i++) msg+=(char)payload[i];
  if (String(topic) == "e4/cmd/brightness")
    ledcWrite(0, constrain(msg.toInt(), 0, 255));
}

bool mqttConnect() {
  if (!wifiReady()) return false;
  mqtt.setServer(mqttBroker.c_str(), 1883);
  mqtt.setCallback(mqttCallback);
  bool ok = mqtt.connect("e4-envmonitor", nullptr, nullptr,
                         "e4/status", 1, true, "offline");
  if (ok) {
    mqtt.publish("e4/status", "online", true);
    char fwMsg[32];
    snprintf(fwMsg, sizeof(fwMsg), "%s/%s", FW_VERSION, FW_DATE);
    mqtt.publish("e4/firmware", fwMsg, true);
    mqtt.subscribe("e4/cmd/#");
  }
  return ok;
}

void mqttUpdate(unsigned long now) {
  if (!wifiReady()) return;
  if (!mqtt.connected()) {
    static unsigned long lastRetry = 0;
    if (now - lastRetry > 5000) { lastRetry = now; mqttConnect(); }
  }
  mqtt.loop();
}

void publishSensors() {
  if (!mqtt.connected()) return;
  char buf[16];
  snprintf(buf,sizeof(buf),"%.1f",ahtTempC);
  mqtt.publish("e4/sensor/temperature",buf,true);
  snprintf(buf,sizeof(buf),"%.1f",ahtHumid);
  mqtt.publish("e4/sensor/humidity",buf,true);
  snprintf(buf,sizeof(buf),"%d",lightPct);
  mqtt.publish("e4/sensor/light",buf,true);
  if (ds18TempC > -90.0f) {
    snprintf(buf,sizeof(buf),"%.1f",ds18TempC);
    mqtt.publish("e4/sensor/probe",buf,true);
  }
}
```

```
// ── Web server ──────────────────────────────────────────────
void setupWebServer() {
  server.on("/", HTTP_GET, [](AsyncWebServerRequest* req) {
    req->send(LittleFS, "/index.html", "text/html");
  });
  server.on("/api/sensors", HTTP_GET, [](AsyncWebServerRequest* req) {
    char json[128];
    snprintf(json, sizeof(json),
      "{\"temp\":%.1f,\"humid\":%.1f,\"probe\":%.1f,\"light\":%d,\"reads\":%ld}",
      ahtTempC, ahtHumid, ds18TempC, lightPct, readCount);
    req->send(200, "application/json", json);
  });
  server.serveStatic("/", LittleFS, "/");
  server.onNotFound([](AsyncWebServerRequest* req) {
    req->send(404,"text/plain","Not found");
  });
  ElegantOTA.begin(&server);
  server.begin();
  Serial.printf("Web: http://%s\n",
                WiFi.localIP().toString().c_str());
}

void initNetwork() {
  Preferences p;
  p.begin("network", true);
  wifiSSID   = p.getString("ssid",   DEFAULT_SSID);
  wifiPass   = p.getString("password",DEFAULT_PASS);
  mqttBroker = p.getString("broker",  DEFAULT_BROKER);
  p.end();
  if (!LittleFS.begin(true))
    Serial.println("LittleFS mount failed.");
  wifiConnect();
}
```

7 Logger.h — SD card data logging

```
// Logger.h — E4 P01 Environmental Monitor
#pragma once
#include <Arduino.h>
#include <SPI.h>
#include <SD.h>

extern float ahtTempC, ahtHumid, ds18TempC;
extern int   lightPct;

#define LOG_FILE   "/ENV_LOG.CSV"
#define MAX_LOG_KB  512

bool sdOk      = false;
bool logEnabled= true;
long logRows   = 0;

// Call AFTER tft.init() — SPI bus must be configured first
bool initLogger() {
  sdOk = SD.begin(SD_CS);
  if (!sdOk) {
    Serial.println("SD mount failed."); return false;
  }
  if (!SD.exists(LOG_FILE)) {
    File f = SD.open(LOG_FILE, FILE_WRITE);
    if (f) {
      f.println("millis,tempAHT,humidity,tempProbe,lightPct");
      f.close();
    }
  } else {
    // Count existing rows
    File f = SD.open(LOG_FILE);
    if (f) {
      while (f.available()) if (f.read()=='\n') logRows++;
      f.close(); logRows--;  // subtract header
    }
  }
  Serial.printf("Logger: SD OK, %ld existing rows\n", logRows);
  return true;
}

bool logReading() {
  if (!sdOk || !logEnabled) return false;
  // Rotate if too large
  File chk = SD.open(LOG_FILE);
  if (chk && chk.size() > (unsigned long)MAX_LOG_KB*1024) {
    chk.close();
    if (SD.exists("/ENV_BAK.CSV")) SD.remove("/ENV_BAK.CSV");
    SD.rename(LOG_FILE, "/ENV_BAK.CSV");
    logRows = 0;
    File f2 = SD.open(LOG_FILE, FILE_WRITE);
    if (f2) { f2.println("millis,tempAHT,humidity,tempProbe,lightPct"); f2.close(); }
  } else if (chk) { chk.close(); }
```

```
  char line[64];
  snprintf(line, sizeof(line), "%lu,%.2f,%.1f,%.2f,%d",
           millis(), ahtTempC, ahtHumid, ds18TempC, lightPct);
  File f = SD.open(LOG_FILE, FILE_APPEND);
  if (!f) return false;
  f.println(line);
  f.close();
  logRows++;
  return true;
}

unsigned long logFileSize() {
  File f = SD.open(LOG_FILE);
  if (!f) return 0;
  unsigned long sz = f.size();
  f.close(); return sz;
}
```

8 main.cpp — application coordinator

```
#include <Arduino.h>
#include <esp_ota_ops.h>
#include "Settings.h"
#include "Sensors.h"
#include "Display.h"
#include "Network.h"
#include "Logger.h"
#include "Button.h"
#include "TonePlayer.h"

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);
Button      nextBtn(BTN_NEXT);
Button      entBtn(BTN_ENT);
TonePlayer  tonePlayer(PIEZO);
SettingsManager settings;

const ToneNote ALERT_SND[] = {{880,80},{0,40},{880,80},{0,40},{880,200}};
const ToneNote STARTUP[]   = {{262,120},{330,120},{392,120},{523,250}};

unsigned long lastSensorRead = 0;
unsigned long lastFooter     = 0;
bool alertActive = false;
bool otaValid    = false;

void checkAlerts() {
  bool triggered = (ahtTempC  > settings.s.tempAlert) ||
                   (ahtHumid  > settings.s.humidAlert);
  if (triggered && !alertActive) {
    alertActive = true;
    digitalWrite(LED_BLUE, HIGH);
    tonePlayer.play(ALERT_SND,
                    sizeof(ALERT_SND)/sizeof(ALERT_SND[0]));
    Serial.printf("ALERT: T:%.1f H:%.1f\n", ahtTempC, ahtHumid);
  }
  if (!triggered) {
    alertActive = false;
    digitalWrite(LED_BLUE, LOW);
  }
}

void handleButtons(Button::Event nextEv, Button::Event entEv) {
  if (nextEv == Button::SHORT_PRESS)
    currentPage = (DisplayPage)((currentPage+1) % PAGE_COUNT);
  if (nextEv == Button::LONG_PRESS) lastSensorRead = 0; // force read
  if (entEv == Button::SHORT_PRESS && alertActive) {
    alertActive = false;
    tonePlayer.stop();
    digitalWrite(LED_BLUE, LOW);
  }
  if (entEv == Button::SHORT_PRESS) logEnabled = !logEnabled;
  if (entEv == Button::LONG_PRESS)  settings.reset();
```

```
}

void setup() {
  Serial.begin(115200);
  Serial.printf("P01 Env Monitor  FW:%s  %s\n",
                FW_VERSION, FW_DATE);

  // Factory reset check
  pinMode(BTN_NEXT, INPUT_PULLUP);
  pinMode(BTN_ENT,  INPUT_PULLUP);
  if (digitalRead(BTN_NEXT)==LOW && digitalRead(BTN_ENT)==LOW) {
    settings.reset();
    Serial.println("Factory reset!");
  }

  settings.load();

  // LED init
  pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
  pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);
  pinMode(LED_BLUE,  OUTPUT); digitalWrite(LED_BLUE,  LOW);

  // TFT init FIRST (before SD)
  pinMode(TFT_BL, OUTPUT); digitalWrite(TFT_BL, LOW);
  tft.init(); tft.setRotation(0); tft.fillScreen(0x0000);  // Landscape 320x240
  ledcSetup(0, 5000, 8); ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, settings.s.brightness);
  spr.setColorDepth(16); spr.createSprite(316, 36);

  // Buttons + audio
  nextBtn.begin(); entBtn.begin(); tonePlayer.begin();

  // Sensors
  sensorsOk = initSensors();
  if (!sensorsOk) digitalWrite(LED_RED, HIGH);

  // SD logger (after TFT)
  initLogger();

  // Network (async)
  initNetwork();

  // Touch calibration
  Preferences p;
  p.begin("touch", true);
  uint16_t cal[5];
  if (p.getBytesLength("cal") == sizeof(cal)) {
    p.getBytes("cal", cal, sizeof(cal));
    tft.setTouch(cal);
  }
  p.end();

  drawChrome();
```

```
  readAllSensors();
  updateLivePage(ahtTempC, ahtHumid, ds18TempC, lightPct,
                 settings.s.tempAlert, settings.s.humidAlert);

  tonePlayer.play(STARTUP, sizeof(STARTUP)/sizeof(STARTUP[0]));
  digitalWrite(LED_GREEN, HIGH);
  Serial.println("P01 ready.");

  // Web server starts after WiFi connects (handled in wifiUpdate)
  // ElegantOTA.onStart/onProgress/onEnd callbacks omitted for brevity
  // See T14 for full OTA progress bar implementation
}

bool webStarted = false;

void loop() {
  unsigned long now = millis();

  wifiUpdate(now);
  mqttUpdate(now);
  ElegantOTA.loop();
  sensorsUpdate(now);
  tonePlayer.update();

  // Start web server once WiFi connects
  if (wifiReady() && !webStarted) {
    setupWebServer(); webStarted = true;
  }

  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();
  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);

  handleButtons(nextEv, entEv);
  bool forceRead = false;
  handleTouch(tx, ty, touched, forceRead, now);

  bool shouldRead = forceRead ||
                    (now - lastSensorRead >= settings.s.logInterval);
  if (shouldRead) {
    lastSensorRead = now;
    readAllSensors();
    checkAlerts();
    updateLivePage(ahtTempC, ahtHumid, ds18TempC, lightPct,
                   settings.s.tempAlert, settings.s.humidAlert);
    logReading();
    publishSensors();
  }

  if (now - lastFooter >= 1000) {
    lastFooter = now;
    updateFooter(now, wifiReady(), mqtt.connected(), logEnabled);
```

```
  }

  if (!otaValid && now > 10000) {
    esp_ota_mark_app_valid_cancel_rollback();
    otaValid = true;
  }
}
```

9 Commissioning checklist

Work through this checklist in order on every new E4 Environmental Monitor build. Do not skip steps.

☐	Step
☐	Format microSD card FAT32. Insert into TFT module slot before powering on.
☐	Upload firmware via USB first (Ctrl+Alt+U). Confirm FW_VERSION in Serial monitor.
☐	Upload filesystem (Upload Filesystem Image). Confirm LittleFS mount in Serial.
☐	On first boot: hold BOOT + ENT simultaneously to trigger factory reset if NVS has stale data.
☐	Confirm Serial shows: AHT20 OK, DS18B20 found, SD OK, LittleFS OK.
☐	Confirm GREEN LED lights at end of setup(). RED LED means sensor init failure.
☐	Startup melody plays from PIEZO — confirms TonePlayer.h and LEDC channel 1.
☐	Run touch calibration: hold ENT long at boot to force recalibration, or run T09 calibration sketch first.
☐	Confirm live readings display on TFT page 1. All three temperatures and light bar visible.
☐	Tap NEXT or touch PAGE button. Confirm page 2 statistics appear.
☐	Tap REFRESH button. Confirm immediate sensor read.
☐	Check Serial output for WiFi connection and IP address.
☐	Open browser at http://<E4-IP>. Confirm web dashboard loads and updates.
☐	Open http://<E4-IP>/update. Confirm ElegantOTA page loads.
☐	Check MQTT Explorer on PC. Confirm e4/status = online and sensor topics updating.
☐	Confirm e4/firmware topic shows correct FW_VERSION/FW_DATE.
☐	In Node-RED: add mqtt in nodes for e4/sensor/# topics. Confirm data flowing.
☐	Power cycle the board. Confirm settings restored from NVS. Boot counter increments.
☐	Perform OTA update: build new firmware, increment FW_VERSION, upload via /update.
☐	Confirm updated FW_VERSION appears in TFT header after reboot.

10 Extending the project

The Environmental Monitor is a solid foundation for more advanced features. Suggested extensions in roughly increasing complexity:

- Add a DS3231 RTC module to the I2C bus (address 0x68) for real timestamped log entries instead of millis()
- Add a second AHT20 or BMP280 on the second I2C expansion header for multi-zone monitoring
- Add a BMP280 for barometric pressure trend arrow (rising/falling) and approximate altitude
- Serve the CSV log file via a /api/log endpoint so it can be downloaded from the web dashboard
- Add a Node-RED flow that emails or sends a push notification when alert thresholds are exceeded
- Store the maximum log interval in NVS and expose it via the web settings API for remote configuration
- Add a Chart.js graph to index.html that shows temperature history from the last N readings

- Migrate to a Tier 3 P02-style architecture with RTC timestamps and daily log rotation